

CURSO TÉCNICO EM INFORMÁTICA

Linguagem de Programação Estruturada

Aula 2 — Sintaxe, Semântica e Dados

Professor: Rafael de Olanda · Turma: TI1F



Agenda da aula 2

- Recap rápido da aula 1
- 1. Sintaxe vs semântica
- 2. Tipos de dados em C
- 3. Variáveis e constantes
- 4. Arrays/vetores — o conceito
- Recap e encerramento

Recap — o que vimos na aula 1

- Algoritmo, abstração, e o caminho até o código: algoritmo → pseudocódigo → fluxograma → código
- Programação estruturada: sequência, decisão e repetição
- Baixo nível vs alto nível, compilado vs interpretado
- C e Pascal: linguagens compiladas que vamos usar nesta disciplina
- Hoje: como o compilador "lê" o que vocês escrevem — sintaxe, semântica e os primeiros tipos de dado



1

Sintaxe vs Semântica

As duas formas de um programa estar "errado"

Testem vocês mesmos, sem instalar nada

Ainda não temos IDE instalada (isso vem só na aula 5) — mas dá pra ver os erros de verdade, agora, no navegador

Para C

onlinegdb.com/online_c_compiler

Cole o código, clique em Run e veja a mensagem de erro real do compilador

Para Pascal

onlinegdb.com/online_pascal_compiler

Mesma ideia — cole, rode, e compare a mensagem de erro com a do C

A partir de agora, todo bloco de código desta aula aparece em C e em Pascal lado a lado — testem os dois e comparem as mensagens de erro que cada compilador dá.

O que é sintaxe?

As regras gramaticais de uma linguagem de programação

Definição: o conjunto de regras que dizem como o código **PRECISA** ser escrito para ser válido

Todo programa tem uma estrutura mínima — e a sintaxe de cada linguagem define exatamente como escrevê-la:

C

```
#include <stdio.h>    // biblioteca

int main() {          // onde tudo começa
    // comandos aqui, cada um com ;
    return 0;
}                      // fecha o bloco
```

Pascal

```
program Ola;           // nome do programa

begin                  // abre o bloco
    // comandos aqui, com ; entre eles
end.                   // fecha o bloco,
                       // com ponto final!
```

Em C: { } abrem/fecham um bloco, e cada comando termina com ;.

Em Pascal: begin/end demarcam o bloco (o último end. leva um ponto final), e ; separa os comandos.

O que é semântica?

O significado por trás do código — mesmo gramaticalmente correto

Definição: se o código faz sentido **LÓGICO**, mesmo estando gramaticalmente correto

Voltando ao português: "O quadrado é redondo" está gramaticalmente perfeito — mas não faz sentido nenhum

Erro de semântica — compila nas duas, mas dividir por zero não faz sentido:

C

```
int main() {  
    int idade = 20;  
    int media = idade / 0;  
    printf("%d", media);  
    return 0;  
}
```

Pascal

```
program Media;  
var idade, media: integer;  
begin  
    idade := 20;  
    media := idade div 0;  
    writeln(media);  
end.
```



teste ao vivo em onlinegdb.com

Exercício — ache o erro

Trecho A é C, Trecho B é Pascal — sintaxe ou semântica em cada um?

Trecho A — C

```
int main() {  
    int x = 10;  
    if (x > 5) {  
        printf("maior");  
    return 0;  
}
```

Trecho B — Pascal

```
program Divisao;  
var a, b, resultado: integer;  
begin  
    a := 10;  
    b := 0;  
    resultado := a div b;  
    writeln(resultado);  
end.
```

Trecho A: falta fechar a chave do if — erro de sintaxe. Trecho B: compila, mas dividir por b (que vale 0) não faz sentido — erro de semântica.



teste ao vivo em onlinegdb.com



2

Tipos de Dados em C

O que uma variável pode guardar, e quanto espaço isso ocupa

Tipos primitivos em C

Tipo	Guarda	Tamanho típico	Exemplo de valor
<code>int</code>	Números inteiros	4 bytes	-3, 0, 42
<code>float</code>	Números decimais (precisão simples)	4 bytes	3.14
<code>double</code>	Números decimais (precisão dupla)	8 bytes	3.14159265
<code>char</code>	Um único caractere	1 byte	'A', '7', '\$'

C usa tipagem estática: você declara o tipo de cada variável, e ele não muda depois — diferente do que vocês verão em linguagens de alto nível como Python ou JavaScript

Exemplo prático — declarando cada tipo

O mesmo programa, em C e em Pascal

C

```
#include <stdio.h>

int main() {
    int idade = 17;
    float altura = 1.72;
    double distancia = 384400.789;
    char inicial = 'R';
    printf("Idade: %d\n", idade);
    printf("Altura: %.2f\n", altura);
    printf("Distancia: %.3f\n", distancia);
    printf("Inicial: %c\n", inicial);
    return 0;
}
```

Pascal

```
program Tipos;
var
    idade: integer;
    altura: single;
    distancia: double;
    inicial: char;
begin
    idade := 17;
    altura := 1.72;
    distancia := 384400.789;
    inicial := 'R';
    writeln('Idade: ', idade);
    writeln('Altura: ', altura:0:2);
    writeln('Distancia: ', distancia:0:3);
    writeln('Inicial: ', inicial);
end.
```

The background is a solid dark blue. There are three large, semi-transparent blue circles of varying shades. One is in the top right corner, another is in the bottom left corner, and a third is partially visible on the left side. The number '3' is centered within a bright blue circle on the left side of the slide.

3

Variáveis e Constantes

Guardando e protegendo valores dentro do programa

Variáveis

Um espaço na memória com nome, tipo e valor

- Declarar: reservar um espaço na memória, com um tipo e um nome
- Atribuir: colocar (ou trocar) o valor guardado ali
- Regras de nomenclatura em C: começar com letra ou `_`, sem espaços, sem acentos, case-sensitive (idade $\neq$ Idade)
- Boas práticas: nomes que dizem o que a variável representa (idade, não x; totalCompras, não tc)
- Em C é possível declarar e atribuir ao mesmo tempo. No Pascal declaração e atribuição ocorrem em blocos separados.

C

```
int idade;           // declaracao
idade = 17;          // atribuicao
int nota = 8;         // declaracao+atribuicao
nota = nota + 1;      // nota vira 9
```

Pascal

```
var idade, nota: integer; // declaracao
begin
    idade := 17; // atribuicao
    nota := 8;
    nota := nota + 1;
```

Constantes

Um valor que não pode mudar depois de definido

const

```
const float PI = 3.14159;
```

O compilador garante: se alguém tentar mudar PI depois, dá erro de compilação

#define

```
#define PI 3.14159
```

O pré-processador troca PI pelo valor antes mesmo de compilar — mais "antigo", mas ainda comum em código C

Em Pascal, a forma padrão é: const PI = 3.14159; — não existe um equivalente direto ao #define do C.

Exemplo prático — área de um círculo

Juntando variável e constante — em C e em Pascal

C

```
#include <stdio.h>

const float PI = 3.14159;

int main() {
    float raio = 4.5;
    float area;

    area = PI * raio * raio;

    printf("Area: %.2f\n", area);
    return 0;
}
```

Pascal

```
program AreaCirculo;
const
    PI = 3.14159;
var
    raio, area: real;
begin
    raio := 4.5;
    area := PI * raio * raio;
    writeln('Area: ', area:0:2);
end.
```



4

Arrays / Vetores

Guardando várias informações do mesmo tipo, organizadas

O que é um array (vetor)?

Uma "fileira" de variáveis do mesmo tipo, acessadas por posição

Em vez de criar nota1, nota2, nota3, nota4, nota5..., um array guarda todas as notas juntas, sob um único nome

Cada posição tem um índice — e em C (como na maioria das linguagens), a contagem começa em 0



notas[2] vale 9 · notas[0] vale 8 · o último índice válido aqui é notas[4] (não notas[5]!)

Exercício rápido

Com base no array de notas do slide anterior

1. Qual é o valor de notas[3]?
2. Qual índice guarda o valor 7?
3. Se eu quisesse guardar mais uma nota (a 6ª), esse array de 5 posições comportaria? O que precisaria mudar?
4. Em quais outras situações podemos usar vetores/arrays?

Recapitulando a aula 2

- Sintaxe: as regras gramaticais da linguagem — erro de sintaxe impede até de compilar
- Semântica: o sentido lógico do código — pode compilar e ainda estar errado
- 4 tipos primitivos em C: int, float, double, char — tipagem estática
- Variável: espaço nomeado que guarda um valor que pode mudar; constante: valor fixo (const ou #define)
- Array: várias variáveis do mesmo tipo, acessadas por índice — começando em 0

Próxima aula

- **Aula 3 — Lógica aplicada (revisão rápida)**
- Estruturas de controle revistas: if / else / switch
- Laços revistos: for, while, do-while
- Do fluxograma ao pseudocódigo estruturado — exercícios em duplas

Obrigado!



Dúvidas? Vamos conversar antes da próxima aula.